OFPPT La Voie de l'Avenir	ROYAUME DU MAROC مكتب التكوين المهني وإنعاش الشغل *Office de la Formation Professionnelle et de la Promotion du Travail*	**Formateur :** ***HAFSA MHAND OUAMMI***
	Filière : DEVOWFS	***Niveau*** *: 2ème Année*
	Module *:* M204 - Développer en FrontEnd	***Année de formation****: 2026/2027*

TP 2 :

L'objectif

- Utilisation de la fonction map
- Utilisation des props
- Utilisation des états locaux (State)
- Utilisation des événements
- Utilisation du formulaire

Styliser l'interface avec CSS ou une bibliothèque comme Bootstrap ou tailwind

Exercice 1 :

Créez un composant avec :

1. Un champ texte contrôlé (lié au state) demandant un pseudo, limité à 20 caractères.
2. Un compteur « X / 20 caractères » sous le champ, qui passe en rouge lorsque la limite est atteinte.
3. Un encadré « Aperçu en direct » qui affiche « Bonjour, [pseudo] ! » et se met à jour à chaque frappe. Si le champ est vide, affichez « Bonjour, anonyme ! ».

Exercice 2 :

1. Créer un composant `StudentProfile`.
2. Déclarer un état `isVisible` initialisé à `false`.
3. Afficher le nom de l'étudiant et sa filière uniquement lorsque `isVisible` vaut `true`.
4. Ajouter un bouton permettant d'afficher ou de masquer les informations.
5. Modifier le texte du bouton selon l'état :
6. « Afficher mon profil » si les informations sont masquées.
7. « Masquer mon profil » si elles sont affichées.
8. Ajouter un message « Les informations sont masquées » lorsque le profil n'est pas visible.

Exercice 3 :

Une boutique souhaite afficher ses produits dans une interface simple. Chaque produit doit être présenté dans une carte réutilisable. Les informations seront stockées dans un tableau d'objets.

On dispose des données suivantes :

```
products = [
  { id: 1, name: "Clavier", price: 150, stock: true },
  { id: 2, name: "Souris", price: 80, stock: true },
  { id: 3, name: "Écran", price: 1200, stock: false },
  { id: 4, name: "Casque", price: 250, stock: true }
];
```

Créer un composant `ProductCard` qui reçoit les informations d'un produit via les props.

1. Afficher le nom et le prix du produit.
2. Afficher « Disponible » si le produit est en stock, sinon « Rupture de stock ».
3. Créer un composant `ProductList` qui parcourt le tableau `products` avec `map()`.
4. Utiliser `ProductCard` pour afficher chaque produit.
5. Utiliser la propriété `id` comme valeur de `key`.
6. Afficher tous les produits dans le composant `App`.

Exercice 4 :

On veut développer une petite interface de gestion de tâches. L'utilisateur pourra ajouter des tâches à une liste et supprimer celles dont il n'a plus besoin.

On dispose des données suivantes :

```
tasks = [

    {
        "id": 5,
        "titre": "Maîtriser UML",
        "statut": "Terminé"
    },
    {
        "id": 7,
        "titre": "Réviser JS",
        "statut": "En cours"
    }
]
```

1. Créez un composant ListeTaches avec deux states : taches (tableau, initialisé avec deux tâches de départ de la forme { id, texte, statut }) et nouvelleTache (chaîne vide).
2. Ajoutez un formulaire avec un champ texte contrôlé et un bouton **Ajouter**. À la soumission, ajoutez une tâche à la liste **sans modifier directement le tableau** (utilisez `[...taches, nouvelleTache]`).
   Ignorez les textes vides (Si la `nouvelleTache est null)` et videz le champ après l'ajout.
3. Créez un composant `Tache` qui reçoit en props `tache`. Il affiche le texte, une case à cocher et un bouton **Supprimer**.
4. Affichez les tâches avec `map` (avec une `key` basée sur l'`id`). Une tâche terminée est barrée (`textDecoration: "line-through"`).
5. Implémentez `basculerTache(id)` (inverse `terminee`)
6. Implémentez `supprimerTache(id)` (utilise `filter`)
7. Affichez sous la liste : « X tâche(s) restante(s) sur Y ». Si la liste est vide, affichez « Aucune tâche pour le moment »

Exercice 5 :

Une application permet à ses utilisateurs de gérer une liste de leurs films préférés. L'utilisateur peut ajouter un film à sa liste et supprimer un film lorsqu'il ne souhaite plus le conserver.

Au démarrage, la liste contient les films suivants :

- Inception
- Interstellar
- The Dark Knight

1. Créer un composant `FavoriteMovies`.
2. Déclarer un état `movieName` pour stocker le nom du film saisi par l'utilisateur.
3. Déclarer un état `movies` initialisé avec les trois films proposés.
4. Créer un formulaire contrôlé contenant un champ de saisie et un bouton « Ajouter ».
5. Utiliser `onChange` pour mettre à jour l'état `movieName` lorsque l'utilisateur saisit un titre.
6. Lors de la soumission du formulaire, vérifier que le champ n'est pas vide.
7. Ajouter le nouveau film à la liste sans modifier directement le tableau existant.
8. Afficher les films à l'aide de `map()`, en attribuant une propriété `key` unique à chaque élément.
9. Ajouter un bouton « Supprimer » à côté de chaque film pour permettre son retrait de la liste.
10. Utiliser `filter()` pour supprimer le film sélectionné.
11. Afficher le message « Votre liste de favoris est vide » lorsque la liste ne contient aucun film.
12. Réinitialiser le champ de saisie après l'ajout d'un film.